

SUMMER 2026



LONGHORN HAPKIT

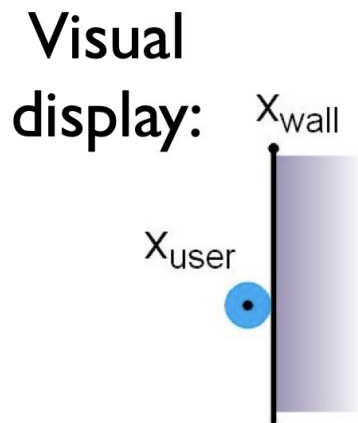
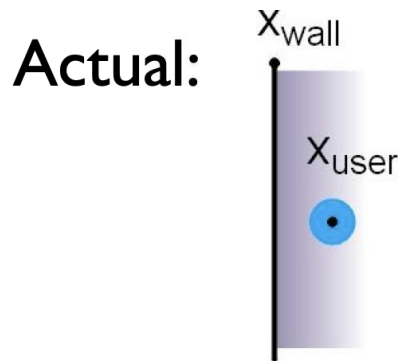
Lab guide #4 - afternoon

ROSS NEUMAN

Assistant Professor of Practice, The University of Texas at Austin

One more way to make a wall better: Visual Feedback of Stiffness

- **trick:** never show the point passing into the surface, even if it is!
- psychophysical studies have shown that this makes the surface appear stiffer to the user



Scripts are uploaded to show all environments we have discussed

- all_environments contains, well, all the environments we've talked about
- Let me know if you have questions about any of them

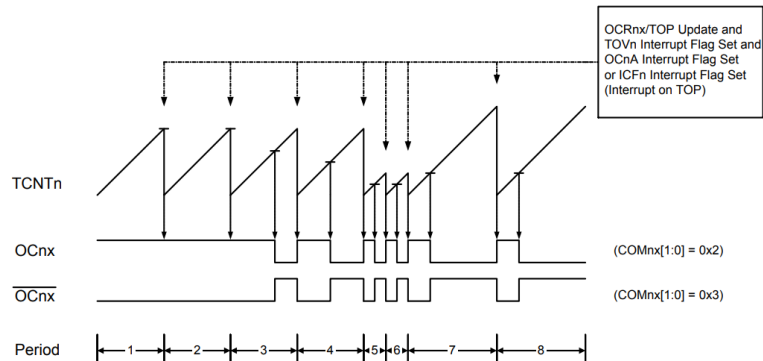
Making your PWM fast and silent

- `all_environments_fast_PWM` has the same environments, but with PWM at 20,000 Hz instead of 490

How did it become quiet?

- Default Arduino PWM frequency on pin 6 is 980Hz
 - This is basically a B5 for music people
 - VERY audible
- Humans typically hear from about 20-20,000Hz
- ATmega328 chip (arduino's brain) is capable of higher PWM
 - Need to talk directly with the **timers**
 - Timer is a hardware counter that increments at a known rate derived from CPU clock
 - CPU clock on arduino is **16MHz (16,000,000 cycles/second)**

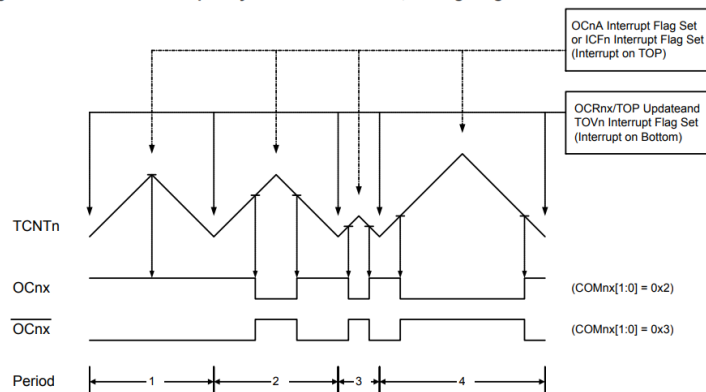
- PWM can be faster and can be **phase and frequency correct** if you address registers on the Atmega chip itself
 - This is like “speaking to the manager” when your Arduino isn’t giving you the help you deserve
- It’s pretty in the weeds, but you can look into the ATmega328P datasheet for more information



ATmega328/P

16-bit Timer/Counter1 (TC1) with PWM

Figure 20-9. Phase and Frequency Correct PWM Mode, Timing Diagram



Talking to the chip

- Rather than using built-in arduino commands, we are talking straight to the MCU onboard
- This mostly consists of setting **registers**

Registers

- Registers in this case are basically 8 bit numbers that tell the MCU different modes to operate in
- We are setting Timer/Counter1 Control Register A or TCCR1A to
- When we type
- $TCCR1A = (1 \ll COM1A1);$
- We are setting the value of TCCR1A to 10000000
- This sets non-inverting PWM

15.11.1 TCCR1A – Timer/Counter1 Control Register A

Bit (0x80)	7	6	5	4	3	2	1	0	
	COM1A1	COM1A0	COM1B1	COM1B0	–	–	WGM11	WGM10	TCCR1A
Read/Write	R/W	R/W	R/W	R/W	R	R	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

• Bit 7:6 – COM1A1:0: Compare Output Mode for Channel A

• Bit 5:4 – COM1B1:0: Compare Output Mode for Channel B

The COM1A1:0 and COM1B1:0 control the output compare pins (OC1A and OC1B respectively) behavior. If one or both of the COM1A1:0 bits are written to one, the OC1A output overrides the normal port functionality of the I/O pin it is connected to. If one or both of the COM1B1:0 bit are written to one, the OC1B output overrides the normal port functionality of the I/O pin it is connected to. However, note that the data direction register (DDR) bit corresponding to the OC1A or OC1B pin must be set in order to enable the output driver.

When the OC1A or OC1B is connected to the pin, the function of the COM1x1:0 bits is dependent of the WGM13:0 bits setting. Table 15-2 shows the COM1x1:0 bit functionality when the WGM13:0 bits are set to a normal or a CTC mode (non-PWM).

Table 15-2. Compare Output Mode, non-PWM

COM1A1/COM1B1	COM1A0/COM1B0	Description
0	0	Normal port operation, OC1A/OC1B disconnected.
0	1	Toggle OC1A/OC1B on compare match.
1	0	Clear OC1A/OC1B on compare match (set output to low level).
1	1	Set OC1A/OC1B on compare match (set output to high level).

Table 15-3 shows the COM1x1:0 bit functionality when the WGM13:0 bits are set to the fast PWM mode.

Table 15-3. Compare Output Mode, Fast PWM⁽¹⁾

COM1A1/COM1B1	COM1A0/COM1B0	Description
0	0	Normal port operation, OC1A/OC1B disconnected.
0	1	WGM13:0 = 14 or 15: Toggle OC1A on compare match, OC1B disconnected (normal port operation). For all other WGM1 settings, normal port operation, OC1A/OC1B disconnected.
1	0	Clear OC1A/OC1B on compare match, set OC1A/OC1B at BOTTOM (non-inverting mode)
1	1	Set OC1A/OC1B on compare match, clear OC1A/OC1B at BOTTOM (inverting mode)

Note: 1. A special case occurs when OCR1A/OCR1B equals TOP and COM1A1/COM1B1 is set. In this case the compare match is ignored, but the set or clear is done at BOTTOM. See Section 15.9.3 “Fast PWM Mode” on page 101 for more details.

Registers

- For TCCR1B we set it to waveform generation mode for **phase and frequency correct PWM** with no prescaling
- Now, OCR1A, which connects to Arduino pin 9, sets the duty cycle
- ICR1 sets the range to 400
- So programming PWM now goes between 0 and 400 instead of 0 and 255

Bot'n'Roll

- We have “botnroll one A” kits if you are tired of the hapkit
- Use in groups or pairs, not enough for everyone
- I have never used them, you're on your own
- https://botnroll.com/onea_en/